OS Orange Problem-2

Name: VIJAY SURIYA P

Section:AIML F

SRN:PES1UG24AM320

Phase 1: Object Storage Foundation

📸 **Screenshot 1A:** Output of ./test_objects showing all tests passing.

```
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check
```

📸 **Screenshot 1B:** find .pes/objects -type f showing the sharded directory structure.

```
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

Phase 2: Tree Objects

📸 **Screenshot 2A:** Output of ./test_tree showing all tests passing.

```
./test_tree
make: 'test_tree' is up to date.
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Phase 3: The Index (Staging Area)

📸 **Screenshot 3A:** Run ./pes init, ./pes add file1.txt file2.txt, ./pes status — show the output.

```
echo "hello" > file.txt
./pes add file.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file.txt
```

📸 **Screenshot 3B:** cat .pes/index showing the text-format index with your entries.

```
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776263502 6 file.txt
```

Phase 4: Commits and History

📸 **Screenshot 4A:** Output of ./pes log showing three commits with hashes, authors, timestamps, and me

```
commit c3d93555ee835801e44c819240fa91266985e1d50658165867339e76a9e43c73
Author: PES User <pes@localhost>
Date:   1776264153

    Commit 33

Commi

commit c9a239536c2b9429a585bd113987faef5c08fb08a01124b77749bb7836e0a124
Author: PES User <pes@localhost>
Date:   1776264153

    Commit 23

Commi

commit 1d6ca6ac3403dec060801c075e8c1faf66764523c378aae7a562fda5d5ca1641
Author: PES User <pes@localhost>
Date:   1776264153

    Commit 1◆
```

📸 **Screenshot 4B:** find .pes -type f | sort showing object store growth after three commits.

```
.pes/HEAD
.pes/index
.pes/objects/1d/6ca6ac3403dec060801c075e8c1faf66764523c378aae7a562fda5d5ca1641
.pes/objects/71/4a8ac694436722923eb810c0211850ce4b6138927afed3f3a92450f585fe42
.pes/objects/a9/3994064c9f0621147b612bdf841da5485745e7024e25ea00e8e38202eb4b30
.pes/objects/b4/ee4550b0b7ad2965e05927480da3474a5b0286971433c4bb24213dcf015713
.pes/objects/b9/ed2198ca3f2ef819afcc21c7f8fef5e60689c2ddfa9bbe6f78f1d88f803edb
.pes/objects/c3/d93555ee835801e44c819240fa91266985e1d50658165867339e76a9e43c73
.pes/objects/c5/55a7730d53c5a8f89e29178f10c81b6202d9dfe23e67dd2331ef963fd3b90c
.pes/objects/c9/a239536c2b9429a585bd113987faef5c08fb08a01124b77749bb7836e0a124
.pes/objects/d4/78b38ba1414b841e3b6c393fd2d355d4fdfc5a38d40a45632113104bc7e874
.pes/refs/heads/main
```

📸 **Screenshot 4C:** cat .pes/refs/heads/main and cat .pes/HEAD showing the reference chain.

```
c3d93555ee835801e44c819240fa91266985e1d50658165867339e76a9e43c73
```

Phase 5 & 6: Analysis-Only Questions
Branching and Checkout

Q5.1 — To implement pes checkout : update .pes/HEAD to point to the branch (ref: refs/heads/), read its commit → tree, walk the tree recursively to write each file to disk, delete files absent from the target tree, and update .pes/index. If the branch doesn't exist yet, create .pes/refs/heads/ with the current HEAD hash first.

Q5.2 — For each IndexEntry, call stat() on the file. If mtime or size differs → file is dirty. Then check if the target branch's tree has a different hash for that path. If both are true → conflict: refuse checkout and report the file

Q5.3 — In detached HEAD, commits are written with HEAD pointing directly to a hash (not a branch ref). Once you switch branches, those commits lose all references and become GC targets. Recovery: note the hash from cat .pes/HEAD, create a new branch file echo "" > .pes/refs/heads/recovery, then echo "ref: refs/heads/recovery" > .pes/HEAD.

Q6.1 — Mark-and-sweep: (Mark) traverse all objects reachable from every branch tip — commits → trees → blobs — and collect hashes into a HashSet. (Sweep) walk every file in .pes/objects/, delete any whose hash is not in the set. Data structure: hash set, O(1) lookup. For 100k commits / 50 branches ≈ 700k objects total → O(700k) linear time, very fast.

Q6.2 — Race: GC finishes marking, then a concurrent commit writes a new blob. GC sweeps and deletes the blob (not in reachable set). The commit then builds a tree referencing the deleted blob → corruption. Real Git fixes this with: (1) a 2-week grace period — never delete objects younger than 2 weeks; (2) reflog — recent history counts as reachable; (3) gc.pid lock — only one GC runs at a time.